21. systemd: Units, Services, Journals

21. systemd: Units, Services, Journals

Phase: Phase 7 — Services and boot
Estimated time: 2 days
Slug: `21-systemd-units`

Learning objectives

- Manage services with systemctl
- Read logs with journalctl
- Write a simple unit file
- Enable services on boot

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

On most modern distributions, systemd starts and supervises your services. If you deploy a web app, a database, or an agent, you will write a unit file, enable it, start it, and read its logs with `journalctl`. The command `systemctl enable --now` is the everyday shortcut that does two things at once: it makes the service start at boot and starts it right now. Mixing up `enable` and `start` is a classic cause of "it worked until the reboot".

Core concepts

Command	Effect
`systemctl start X`	Start now; does not persist across reboot
`systemctl stop X`	Stop now
`systemctl enable X`	Create boot symlink; does not start now
`systemctl enable --now X`	Enable and start: start-and-boot
`systemctl disable X`	Remove boot link
`systemctl restart X`	Stop then start
`systemctl reload X`	Re-read config without full restart
`systemctl status X`	State, recent logs, main PID
`systemctl daemon-reload`	Re-read unit files after editing
`journalctl -u X`	Logs for the unit

Unit file locations:

Path	Use
`/usr/lib/systemd/system/`	Packages ship units here
`/etc/systemd/system/`	Your units and overrides
`/etc/systemd/system/X.service.d/`	Drop-in overrides

Two independent switches:

```
                running now?        starts at boot?
                ------------        ---------------
  start / stop       yes/no              unchanged
  enable / disable   unchanged           yes/no
  enable --now       yes                 yes     (= "start-and-boot")

  unit file --> [Unit] After=, Wants=
            --> [Service] ExecStart=, Restart=, User=
            --> [Install] WantedBy=multi-user.target
                                ^ enable links the unit into this target
```

Worked examples

A minimal service unit:

```
sudo tee /etc/systemd/system/hello.service >/dev/null <<'EOF'
[Unit]
Description=Hello demo service
After=network.target

[Service]
ExecStart=/usr/bin/python3 -m http.server 8099
Restart=on-failure
User=nobody

[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload
```

Enable and start in one step:

```
sudo systemctl enable --now hello.service
systemctl is-enabled hello.service
systemctl is-active hello.service
```

Sample output:

```
Created symlink /etc/systemd/system/multi-user.target.wants/hello.service
enabled
active
```

Status view:

```
systemctl status hello.service --no-pager
```

Sample output:

```
* hello.service - Hello demo service
     Loaded: loaded (/etc/systemd/system/hello.service; enabled; preset: enabled)
     Active: active (running) since Wed 2026-10-21 10:30:00 UTC; 5s ago
   Main PID: 3120 (python3)
```

Logs:

```
journalctl -u hello.service -n 3 --no-pager
journalctl -u hello.service -f
journalctl -b -p err --no-pager | tail -3
journalctl --since "10 min ago" -u hello.service
```

Sample output:

```
Oct 21 10:30:00 web01 systemd[1]: Started hello.service - Hello demo service.
```

Drop-in override and clean-up:

```
sudo systemctl edit hello.service
sudo systemctl disable --now hello.service
```

Helper from the exercise: `enable_now_meaning() { echo start-and-boot; }`

Mini-lab

1. Create `hello.service` as shown and run `daemon-reload`.
2. `systemctl start hello` and `systemctl is-enabled hello` (disabled).
3. Reboot-test thinking: explain why it will not return after reboot.
4. `systemctl enable hello`, then `systemctl is-enabled hello`.
5. Stop it and run `systemctl enable --now hello` to see both effects.
6. Break it: change `ExecStart` to a wrong path, `daemon-reload`, restart, then read `systemctl status` and `journalctl -u hello -n 20`.
7. Kill the process with `sudo kill -9 MAINPID` and watch `Restart=` revive it.
8. Remove it with `disable --now`, delete the file, `daemon-reload`.

Decision guide / failure modes

Symptom	Cause	Fix
Service gone after reboot	Started but not enabled	`enable`
Edits have no effect	Forgot reload	`systemctl daemon-reload`
`status=203/EXEC`	Bad `ExecStart` path or not executable	Fix path, `chmod +x`
`status=200/CHDIR` or 217/USER	Missing directory or user	Create it
Restart loop	App crashes	`journalctl -u X`, add `RestartSec`
Starts too early	Needs network or DB	`After=` and `Wants=`

Use `restart` for code changes, `reload` only if the app supports it, and `enable --now` for first installs. Do not edit packaged units in `/usr/lib`; use `systemctl edit` drop-ins.

Common mistakes

- kill –9 instead of systemctl stop
- Ignoring Restart= policy
- Not checking journal after failed start

Exercises

Work in `exercises/21-systemd-units/`. Read that folder's `README.md` fully before coding.

```
cd exercises/21-systemd-units
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement enable_now_meaning in exercise.sh. It takes no arguments and prints a short token that states what systemctl enable –now does. The token is two ideas joined with hyphens: it starts the service immediately and it also configures it to start at boot. The test compares the output exactly, so print only that token in lowercase. Test your understanding on a VM with a harmless unit by checking systemctl is-active and systemctl is-enabled before and after.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/21-systemd-units/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. enable vs start?
2. How follow service logs?
3. WantedBy meaning?

Further reading

- https://www.freedesktop.org/software/systemd/man/systemctl.html